ИТМО

LLVMad

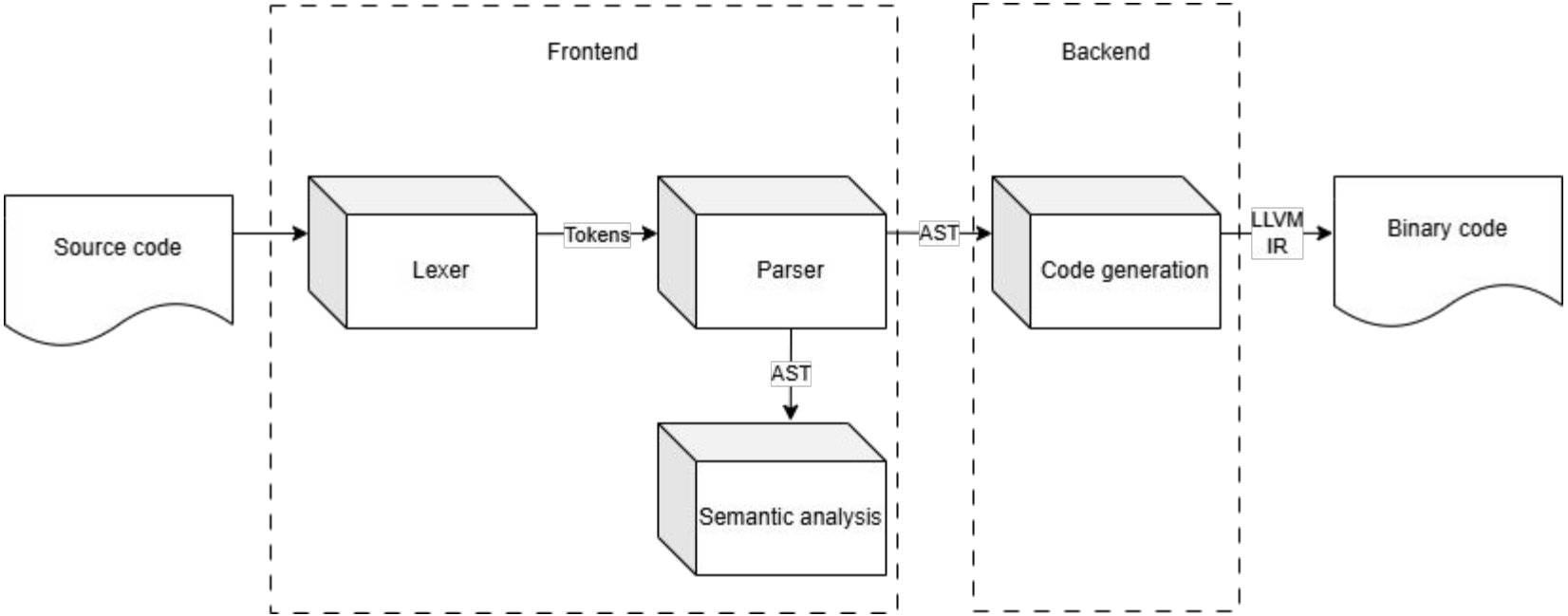
Сафонов Артем
Колесникова Анастасия

Описание языка

- ООП
- Язык реализации компилятора – C++
- Встроенные типы: integer, bool, real, string *(self-made)*, byte
- Цикл for
- Конструкции if/elif, return
- Операторы как методы
- Наследование, override
- Массивы

```
1  module helloworld
2
3  class Main is
4    this() is
5      printf("Hello, world!\n")
6    end
7  end
```

Схема решения



Лексер

```cpp
enum TokenKind {
  TOKEN_EOF,
  TOKEN_IDENTIFIER,  // user-defined: letter { lettter | digit }
  TOKEN_CLASS,       // class
  TOKEN_EXTENDS,     // extends
  TOKEN_VAR_DECL,    // var
  TOKEN_SELFREF,     // this (do not confuse with constructor `this(...)`)
  TOKEN_RETURN,      // return
  TOKEN_MODULE_DECL, // * module
  TOKEN_MODULE_IMP,  // * import
  TOKEN_IF,          // if
  TOKEN_ELSE,        // else
  TOKEN_THEN,        // then
  TOKEN_SWITCH,      // * switch
  TOKEN_CASE,        // * case
  TOKEN_DEFAULT,     // * default
  TOKEN_WHILE,       // while
  TOKEN_LOOP,        // loop
  TOKEN_METHOD,      // method
  TOKEN_FUNC,        // * func
  TOKEN_FOR,         // * for
  TOKEN_STATIC,      // * static
  TOKEN_BBEGIN,      // is
  TOKEN_BEND,        // end
  TOKEN_INT_NUMBER,
```

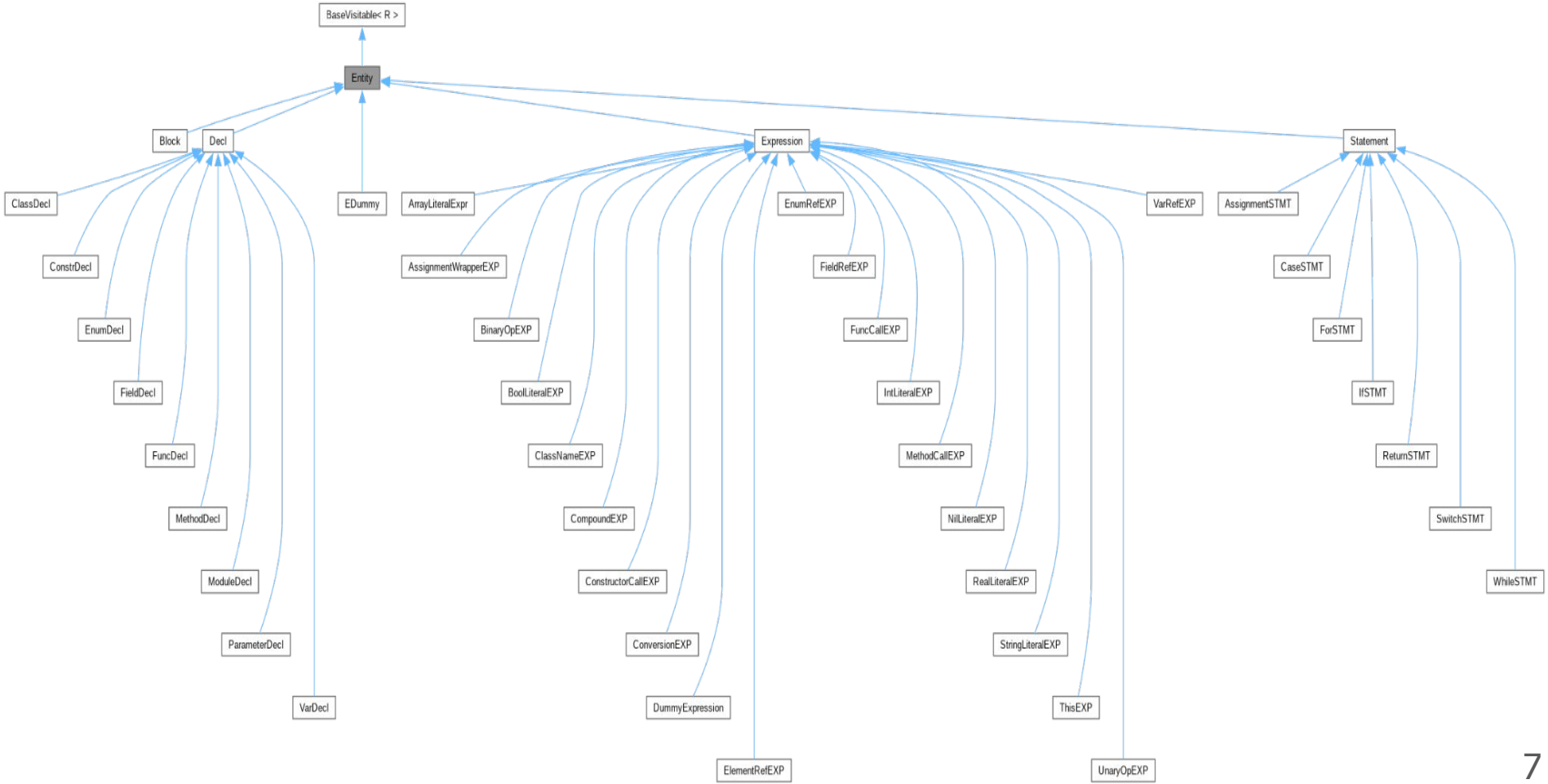
```cpp
enum StateType {
  STATE_NONE = -1,
  STATE_START,
  STATE_READ_WORD,
  STATE_READ_NUM,
  STATE_READ_DECL,
  STATE_READ_IDENT,
  STATE_READ_REAL,
  STATE_READ_ASSIGN,
  STATE_READ_STRING,
  STATE_READ_ARROW,
  STATE_FAIL = 9,
};

class Token {
public:
  TokenKind kind;
  std::variant<std::monostate, int, double, std::string> value;
  size_t line;
  size_t column;

  // Mostly single-character and/or special symbols
  Token(TokenKind kind, size_t line, size_t column)
      : kind(kind), line(line), column(column) {};

  // Int value
  Token(TokenKind kind, int intValue, size_t line, size_t column)
      : kind(kind), value([&] intValue), line(line), column(column) {};

  // Real number
  Token(TokenKind kind, double realValue, size_t line, size_t column)
      : kind(kind), value([&] realValue), line(line), column(column) {};

  // Identifier or string literal
  Token(TokenKind kind, const std::string &lexem, size_t line, size_t column)
      : kind(kind), value(lexem), line(line), column(column) {};
};
```

Лексер

```
switch (curr_state) {
case STATE_START: {
  if (std::isalpha(c) || c == '_') {
    curr_state = STATE_READ_WORD;
  } else if (std::isdigit(c) /*|| c == '-'*/) {
    curr_state = STATE_READ_NUM;
  } else if (std::isspace(c)) {
    curr_state = STATE_START;
  } else if (c == '\n' || c == '\r') {
    curr_state = STATE_START;
  } else if (c == ':') {
    curr_state = STATE_START;
    advance();
    if (peek() == '=') {
      advance();
      return std::make_unique<Token>(TOKEN_ASSIGNMENT, curr_line,
                                     curr_column);
    }

    if (peek() == ':') {
      advance();
      return std::make_unique<Token>(TOKEN_DOUBLE_COLON, curr_line,
                                     curr_column);
    }
```

Парсер

```
83
84  std::shared_ptr<ReturnSTMT> parseReturnStatement();
85
86  std::shared_ptr<SwitchSTMT> parseSwitch();
87
88  std::shared_ptr<CaseSTMT> parseCase();
89
90  std::shared_ptr<VarDecl> parseVarDecl();
91
92  std::shared_ptr<ConversionEXP> parseConversionOperator(std::shared_ptr<Expression> from);
93
94  std::shared_ptr<ConstrDecl> parseConstructorDecl();
95
96  std::shared_ptr<MethodDecl> parseMethodDecl();
97
98  std::shared_ptr<FuncDecl> parseFunctionDecl();
99
100 std::shared_ptr<ClassDecl> parseClassDecl();
101
102 std::shared_ptr<FieldDecl> parseFieldDecl(size_t index = 0);
```


BaseVisitable< R >
Entity
Block
Decl
Expression
Statement
ClassDecl
ConstrDecl
EnumDecl
FieldDecl
FuncDecl
MethodDecl
ModuleDecl
ParameterDecl
VarDecl
EDummy
ArrayLiteralExpr
AssignmentWrapperEXP
BinaryOpEXP
BoolLiteralEXP
ClassNameEXP
CompoundEXP
ConstructorCallEXP
ConversionEXP
DummyExpression
ElementRefEXP
EnumRefEXP
FieldRefEXP
FuncCallEXP
IntLiteralEXP
MethodCallEXP
NilLiteralEXP
RealLiteralEXP
StringLiteralEXP
ThisEXP
UnaryOpEXP
VarRefEXP
AssignmentSTMT
CaseSTMT
ForSTMT
IfSTMT
ReturnSTMT
SwitchSTMT
WhileSTMT

```cpp
enum Ekind {
  // Declarations
  E_Class_Decl,
  E_Constructor_Decl,
  E_Variable_Decl,
  E_Parameter_Decl,
  E_Field_Decl,
  E_Function_Decl,
  E_Method_Decl,
  E_Array_Decl,
  E_List_Decl,
  E_Module_Decl,
  E_Enum_Decl,
  E_Main_Decl,

  // Type related entities
  E_Integer_Type,
  E_Real_Type,
  E_String_Type,
  E_Boolean_Type,
  E_Function_Type,
  E_Opaque_Type,

  // Expression related
  E_Integer_Literal,
  E_Real_Literal,
  E_String_Literal,
  E_Array_Literal,
  E_Var_Reference,
  E_Field_Reference,
  E_Boolean_Literal,
  E_Function,
```

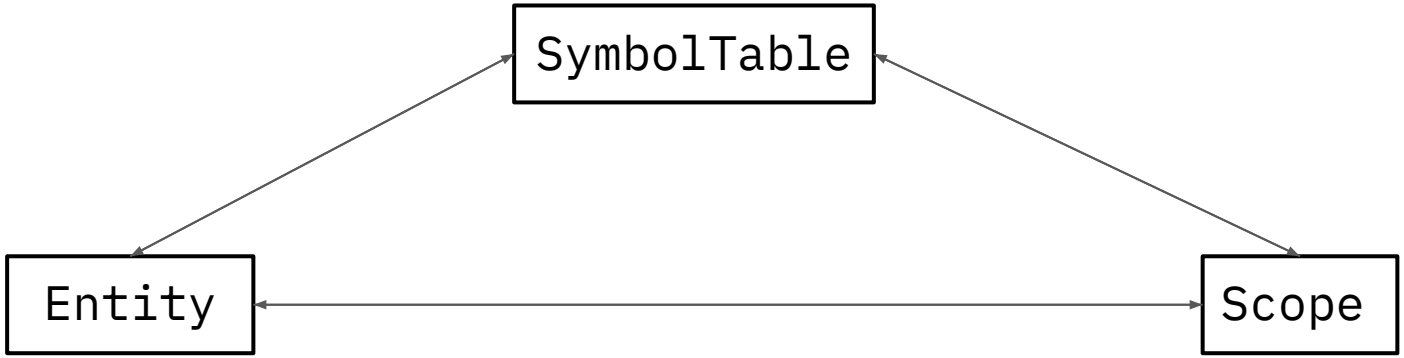
```cpp
/**
 * Foundation for obewrong entities
 */
class Entity : public BaseVisitable<> {
public:
  virtual ~Entity() = default;
  explicit Entity(Ekind kind) : kind(kind), name(), location() {}

  explicit Entity(Ekind kind, std::string name)
  : kind(kind), name( str: std::move(name)), location() {}

  Ekind getKind() const { return kind; };
  Loc getLoc() const { return location; };
  std::string getName() const { return name; };
  void appendToName(const std::string &name) { this->name += name;
  void setName(std::string name) { this->name = name; };

  virtual std::shared_ptr<Type> resolveType(
    const TypeTable &typeTable,
    const std::shared_ptr<Scope<Entity>> &currentScope) = 0;

  virtual bool validate() = 0;

  // STRUCTURAL LINK
  // children nodes
  std::shared_ptr<Entity> next;

  DEFINE_VISITABLE()

protected:
  Ekind kind;
  std::string name;
  Loc location;
};
```

Парсер

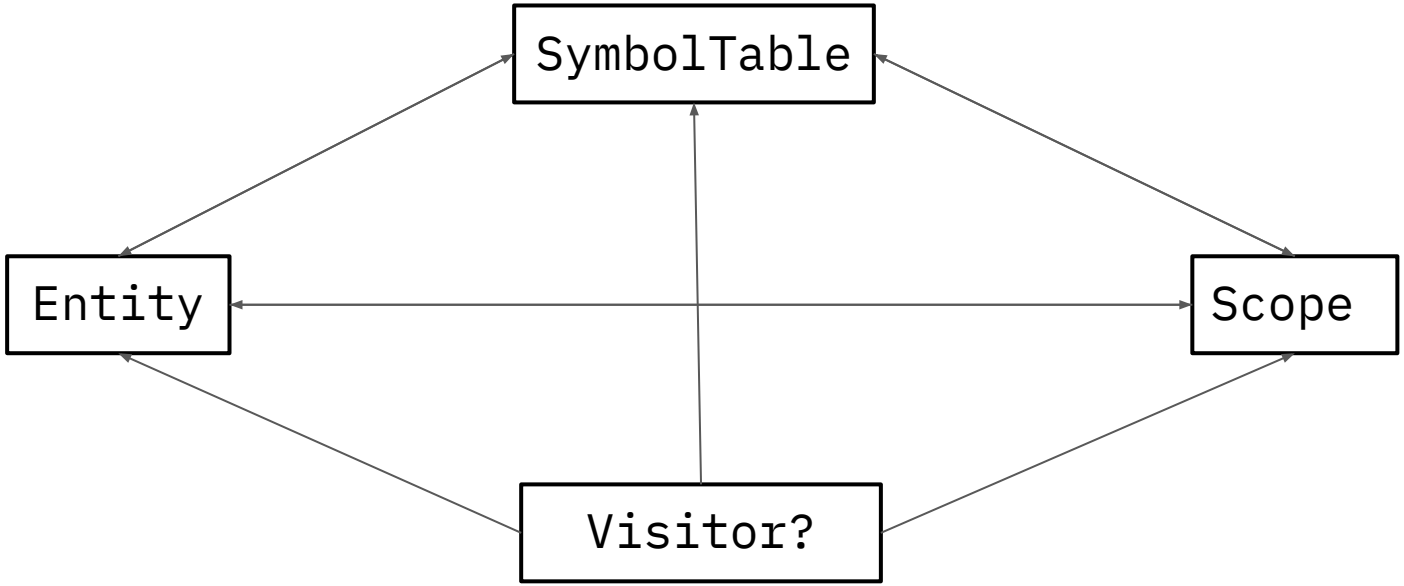
```
ModuleDecl: simpleclass
  ClassDecl: MyClass
    FieldDecl: value
    ConstrDecl: MyClass_CreateInteger
      ParameterDecl: thisMyClass
      ParameterDecl: p
      Assignment
        FieldRef: value
          This
        VarRef: p
    MethodDecl: MyClass_getValue
      ParameterDecl: thisMyClass
      Return
        FieldRef: value
          This
  ClassDecl: Main
    ConstrDecl: Main_Create
      ParameterDecl: thisMain
      VarDecl: obj
        ConstructorCall
          ClassName: MyClass
          MethodCall: Plus
            IntLiteral: 42
            IntLiteral: 2
      VarDecl: result
      Assignment
        VarRef: result
        MethodCall: getValue
          VarRef: obj
      FuncCall: printf
        StringLiteral: result: %d\n
        VarRef: result
```

```
480 std::shared_ptr<IfSTMT> Parser::parseIfStatement() {
481     std::unique_ptr<Token> token = peek();
482     if (token == nullptr || token->kind != TOKEN_IF)
483         return nullptr;
484     token = next(); // eat 'if'
485
486     globalSymbolTable->enterScope( kind: SCOPE_LOOP,  name: "if_else");
487
488     auto condition :shared_ptr<Expression>  = parseExpression();
489
490     auto ifTrue :shared_ptr<Block>  = parseBlock(BLOCK_IN_IF);
491
492     std::shared_ptr<Entity> ifFalse = nullptr;
493     token = peek();
494     if (token->kind == TOKEN_ELSE) {
495         if (peek( i: 2)->kind == TOKEN_IF) {
496             token = next();
497             ifFalse = parseIfStatement();
498         } else {
499             ifFalse = parseBlock(BLOCK_IN_IF);
500         }
501
502         globalSymbolTable->exitScope();
503
504         return std::make_shared<IfSTMT>(condition, ifTrue, ifFalse);
505     }
506
507     globalSymbolTable->exitScope();
508
509     return std::make_shared<IfSTMT>(condition, ifTrue);
510 }
```

СКОУПИНГ!



СКОУПИНГ!



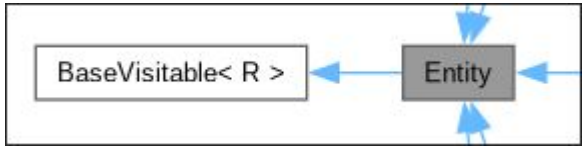
Скоупинг

```cpp
template<typename T>
struct SymbolInfo {
  std::shared_ptr<T> decl; // for parsing phase
  llvm::AllocaInst* alloca;   // for codegen
  bool isInitialized;
};
```

```cpp
template<typename T>
class Scope : public std::enable_shared_from_this<Scope<T>> {
public:
  Scope(ScopeKind kind, const std::string &name, std::weak_ptr<Scope> parent
      : external(false), kind(kind), name( name), parent(parent), depth(

  /**
   * @phase Syntax analysis \n
   * Puts a Declaration info of a certain construct into a map
   * @param name
   * @param decl Declaration info
   */
  template<typename U>
  bool addSymbol(const std::string &name, std::shared_ptr<U> decl) {
    static_assert(std::derived_from<U, T>, "Must be derived from Entity");
    symbols[name].decl = decl;
    return true;
  }

  /**
   * @phase Code generation \n
   * Puts an allocation intance of a variable into a map
   * with previous Decl set
   * @param name
   * @param alloca instance of allocation
   */
  bool addSymbol(const std::string &name, llvm::AllocaInst *alloca) {
    if (!symbols.contains(name))
      return false;
    symbols[name].alloca = alloca;
    return true;
  }
```

```cpp
private:
 ScopeKind kind;
 std::string name;
 std::weak_ptr<Scope> parent;
 std::vector<std::shared_ptr<Scope>> children;
 std::unordered_map
      <std::string, SymbolInfo<T>> symbols;

 int depth;
```

Визиторинг

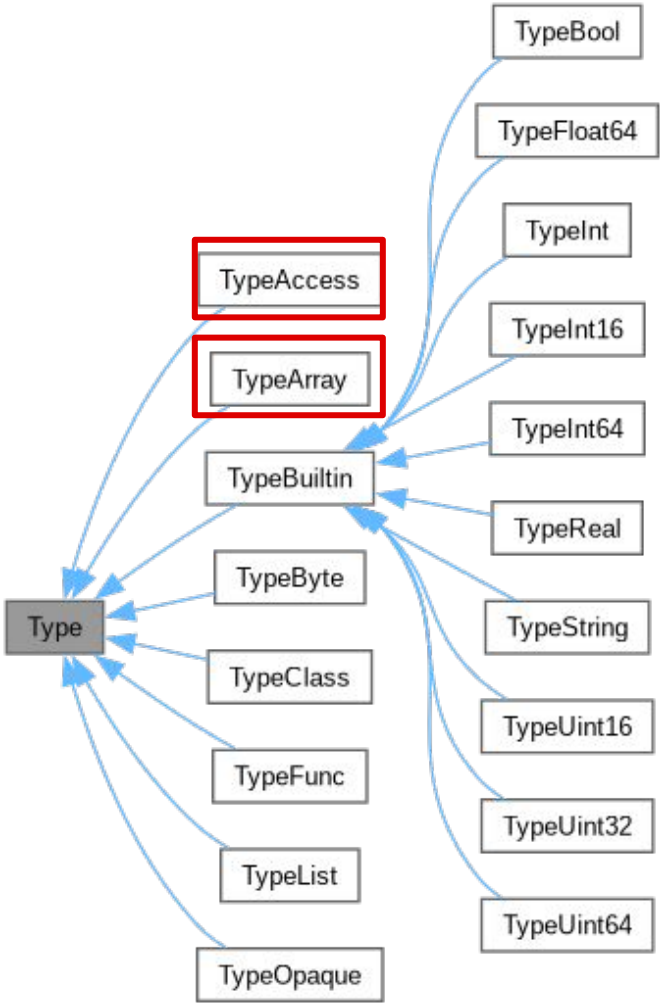



```cpp
class PrinterAst : public BaseVisitor,
                   public Visitor<Entity, void>,
                   public Visitor<Block, void>,
                   public Visitor<EDummy, void>,
                   public Visitor<Statement, void>,
                   public Visitor<AssignmentSTMT, void>,
                   public Visitor<ReturnSTMT, void>,
                   public Visitor<IfSTMT, void>,
                   public Visitor<CaseSTMT, void>,
                   public Visitor<SwitchSTMT, void>,
                   public Visitor<WhileSTMT, void>,
```

```cpp
#ifndef OBW_VISITOR_H
#define OBW_VISITOR_H

// Visitor part
class BaseVisitor {
public:
  virtual ~BaseVisitor() {}
};

template <class T, typename R = void> class Visitor {
public:
  typedef R ReturnType; // Available for clients
  virtual ReturnType visit(T &) = 0;
};

// Visitable part
template <typename R = void> class BaseVisitable {
public:
  typedef R ReturnType;
  virtual ~BaseVisitable() {}
  virtual R accept(BaseVisitor &) = 0;

protected:
  template <class T>
  static ReturnType acceptImpl(T &visited, BaseVisitor &guest)
    // Apply the Acyclic Visitor
    if (Visitor<T> *p = dynamic_cast<Visitor<T> *>(&guest)) {
      return p->visit([&] visited);
    }
    return ReturnType();
  }
};

#define DEFINE_VISITABLE()
  virtual ReturnType accept(BaseVisitor &guest) {
    return acceptImpl(*this, guest);
  }

#endif  #ifndef OBW_VISITOR_H
```

Тайпинг

```
324  class TypeClass : public Type {
325  public:
326    std::shared_ptr<TypeClass> base_class;
327    std::vector<std::shared_ptr<Type>> fields_types;
328    std::vector<std::shared_ptr<TypeFunc>> methods_types;
329
330    TypeClass(const std::string &name,
331              std::vector<std::shared_ptr<Type>> fields_types,
332              std::vector<std::shared_ptr<TypeFunc>> methods_types)
333        : Type( kind: TYPE_CLASS,  name), fields_types(std::move(fields_types)),
334          methods_types(std::move((methods_types))) {}
335
336    std::shared_ptr<TypeFunc> getMethod(const std::string &name) const {
337      for (const auto &method :const shared_ptr<TypeFunc>&  : methods_types) {
338        if (method->name == name)
339          return method;
340      }
341      return nullptr;
342    }
343
344    llvm::Type *toLLVMType(llvm::LLVMContext &lc) override {
       return llvm::StructType::getTypeByName( [&] lc,  Name: llvm::StringRef(name));
346    }
347
348    ~TypeClass() override = default;
```



Реализация семантического анализа

а. Проверка типов

```
void visit(IntLiteralEXP &expr) override {
  auto type = expr.resolveType(globalTypeTable->types[currentModuleName],
                               currentScope);
  if (!type || type->name != "Integer") {
    reportError("Invalid integer literal type", expr.getLoc());
  }
}
```

б. Области видимости

```
  auto [paramDecl, alloc, isInited] =
      *currentScope->getSymbol(decl.getName());
  if (isInited) {
    reportError("Duplicate parameter name: " + decl.getName(), decl.getLoc());
  }
```

в. Существование символа

```
void visit(VarRefEXP &expr) override {
  auto [decl, alloc, isInited] = *currentScope->getSymbol(expr.getName());
  if (!decl) {
    reportError("Undefined variable '" + expr.getName() + "'", expr.getLoc());
  }
}
```

г. Проверка полей и конструкторов класса

```
void visit(ClassNameEXP &expr) override {
  auto classType = globalTypeTable->getType(
      symbolTable->getCurrentScope()->getName(), expr.getName());
  if (!classType) {
    reportError("Undefined class name: " + expr.getName(), expr.getLoc());
    return;
  }
```

Семантический анализ

```
1   module simple
2
3   class Simple is
4     method add(a : Integer) : Integer is
5       var b : Integer
6       b := a + "2"          // Ошибка 1: сложение int и string
7       var i : Integer := 10
8
9       while i loop          // Ошибка 2: условие цикла должно быть bool
10        i := i.Minus(1)
11      end
12
13      printf("res = %d\n", i)
14
15      return "result"       // Ошибка 3: возврат string вместо int
16    end
17  end
```

```
ModuleDecl: simple
  ClassDecl: Simple
    MethodDecl: add
      ParameterDecl: this
      ParameterDecl: a
      VarDecl: b
      Assignment
        VarRef: b
        BinaryOp
          VarRef: a
          StringLiteral: 2
      VarDecl: i
        IntLiteral: 10
      While
        VarRef: i
        Assignment
          VarRef: i
          MethodCall: Minus
            VarRef: i
            IntLiteral: 1
      FuncCall: printf
        StringLiteral: res = %d\n
        VarRef: i
      Return
        StringLiteral: result
(Line: 7, Col: 8)[Semantic Error] Type mismatch for operator +
(Line: 11, Col: 12)[Semantic Error] While loop condition must be boolean
(Line: 17, Col: 20)[Semantic Error] Return type mismatch: expected 'Integer'
```

Генерация кода

```
38  class CodeGenVisitor : public BaseVisitor,
39                         public Visitor<Entity, void>,
40                         public Visitor<Block, void>,
41                         public Visitor<EDummy, void>,
42                         public Visitor<Statement, void>,
43                         public Visitor<AssignmentSTMT, void>,
44                         public Visitor<ReturnSTMT, void>,
45                         public Visitor<IfSTMT, void>,
46                         public Visitor<CaseSTMT, void>,
47                         public Visitor<SwitchSTMT, void>,
48                         public Visitor<WhileSTMT, void>,
49                         public Visitor<ForSTMT, void>,
50                         public Visitor<Expression, void>,
51                         public Visitor<IntLiteralEXP, void>,
52                         public Visitor<RealLiteralEXP, void>,
53                         public Visitor<StringLiteralEXP, void>,
54                         public Visitor<BoolLiteralEXP, void>,
55                         public Visitor<ArrayLiteralExpr, void>,
56                         public Visitor<VarRefEXP, void>,
57                         public Visitor<FieldRefEXP, void>,
58                         public Visitor<ElementRefEXP, void>,
59                         public Visitor<MethodCallEXP, void>,
60                         public Visitor<FuncCallEXP, void>,
61                         public Visitor<ClassNameEXP, void>,
62                         public Visitor<ConstructorCallEXP, void>,
63                         public Visitor<CompoundEXP, void>,
64                         public Visitor<ThisEXP, void>,
65                         public Visitor<ConversionEXP, void>,
66                         public Visitor<BinaryOpEXP, void>,
67                         public Visitor<UnaryOpEXP, void>,
68                         public Visitor<EnumRefEXP, void>,
69                         public Visitor<AssignmentWrapperEXP, void>,
70                         public Visitor<Decl, void>,
71                         public Visitor<FieldDecl, void>,
72                         public Visitor<VarDecl, void>,
73                         public Visitor<ParameterDecl, void>,
74                         public Visitor<MethodDecl, void>,
75                         public Visitor<ConstrDecl, void>,
```

```
749  void CodeGenVisitor::visit(VarDecl &node) {
750      llvm::Function *function = builder->GetInsertBlock()->getParent();
751
752      std::string var_name = node.getName();
753      auto varType = node.type->toLLVMType( [&] *this->context);
754      auto initializer :shared_ptr<Expression> = node.initializer;
755      llvm::AllocaInst *alloca;
756      llvm::Value* initVal;
757      if (initializer) {
758          initializer->accept( [&] *this);
759          initVal = lastValue;
760          if (initializer->getKind() != E_Constructor_Call && initializer->getKind() != E_Array_L
761
762              // if initVal is a pointer (EL_REF, GEP)
763              llvm::Value* initValUnwrap = unwrapPointerReference( node: initializer.get(), initVal);
764
765              alloca = builder->CreateAlloca(varType, ArraySize: nullptr, var_name);
766              builder->CreateStore(initValUnwrap, Ptr: alloca);
767          } else {
768              // For constructor calls, we already have the allocation
769              alloca = dyn_cast<llvm::AllocaInst>(initVal);
770          }
771      }
772      else {
773          alloca = builder->CreateAlloca(varType, ArraySize: nullptr, var_name);
774      }
775
776      currentScope->addSymbol(var_name, alloca);
777  }
```

Генерация кода

```
1   module point
2
3   class Point is
4     var x : Integer
5     var y : Integer
6
7     this() is
8       this.x := 0
9       this.y := 0
10    end
11
12    this(x : Integer, y : Integer) is
13      this.x := x
14      this.y := y
15    end
16
17    method xByy() : Integer is
18      var _x : Integer := this.x
19      var _y : Integer := this.y
20      return _x.Mult(_y)
21    end
22  end
```

```
1   %Point = type { i32, i32 }
2
3   define void @Point_Create(ptr %thisPoint) {
4   entry:
5     %thisPoint1 = alloca ptr, align 8
6     store ptr %thisPoint, ptr %thisPoint1, align 8
7     %0 = load ptr, ptr %thisPoint1, align 8
8     %x = getelementptr inbounds nuw %Point, ptr %0, i32 0, i32 0
9     store i32 0, ptr %x, align 4
10    %1 = load ptr, ptr %thisPoint1, align 8
11    %y = getelementptr inbounds nuw %Point, ptr %1, i32 0, i32 1
12    store i32 0, ptr %y, align 4
13    ret void
14  }
15
16  define void @Point_CreateIntegerInteger(ptr %thisPoint, i32 %x, i32 %y) {
17  entry:
18    %y3 = alloca i32, align 4
19    %x2 = alloca i32, align 4
20    %thisPoint1 = alloca ptr, align 8
21    store ptr %thisPoint, ptr %thisPoint1, align 8
22    store i32 %x, ptr %x2, align 4
23    store i32 %y, ptr %y3, align 4
24    %0 = load ptr, ptr %thisPoint1, align 8
25    %x4 = getelementptr inbounds nuw %Point, ptr %0, i32 0, i32 0
26    %x5 = load i32, ptr %x2, align 4
27    store i32 %x5, ptr %x4, align 4
28    %1 = load ptr, ptr %thisPoint1, align 8
29    %y6 = getelementptr inbounds nuw %Point, ptr %1, i32 0, i32 1
30    %y7 = load i32, ptr %y3, align 4
31    store i32 %y7, ptr %y6, align 4
32    ret void
33  }
```

Генерация кода

```
1   module simpleclass
2
3   class MyClass is
4       var value : Integer
5
6       this(p: Integer) is
7           this.value := p
8       end
9
10      method getValue() : Integer is
11          return this.value
12      end
13  end
14
15  class Main is
16      this() is
17          var obj : MyClass(42.Plus(2))
18          var result : Integer
19          result := obj.getValue()
20          printf("result: %d\n", result)
21      end
22  end
```

```
4   %MyClass = type { i32 }
5   %Main = type {}
6
7   @0 = private unnamed_addr constant [12 x i8] c"result: %d\0A\00", align 1
8
9   declare i32 @printf(ptr, ...)
10
11  declare ptr @malloc(i64)
12
13  declare void @free(ptr)
14
15  declare i64 @strlen(ptr)
16
17  declare ptr @strcpy(ptr, ptr)
18
19  define void @MyClass_CreateInteger(ptr %thisMyClass, i32 %p) {
20  entry:
21    %p2 = alloca i32, align 4
22    %thisMyClass1 = alloca ptr, align 8
23    store ptr %thisMyClass, ptr %thisMyClass1, align 8
24    store i32 %p, ptr %p2, align 4
25    %0 = load ptr, ptr %thisMyClass1, align 8
26    %value = getelementptr inbounds nuw %MyClass, ptr %0, i32 0, i32 0
27    %p3 = load i32, ptr %p2, align 4
28    store i32 %p3, ptr %value, align 4
29    ret void
30  }
31
32  define i32 @MyClass_getValue(ptr %thisMyClass) {
33  entry:
34    %thisMyClass1 = alloca ptr, align 8
35    store ptr %thisMyClass, ptr %thisMyClass1, align 8
36    %0 = load ptr, ptr %thisMyClass1, align 8
37    %value = getelementptr inbounds nuw %MyClass, ptr %0, i32 0, i32 0
38    %1 = load i32, ptr %value, align 4
39    ret i32 %1
40  }
```

Дополнения

```
module test

import string

class Main is
  this() is

    var str : String := String("Hello")

    printf("str = %s | length = %d\n", str.data,

    str.append('!')

    printf("str = %s | length = %d\n", str.data,

  end
end
```

1. Libc:
   - malloc
   - free
   - strlen
   - strcpy
2. Access types
3. For loops
4. Module system

```
module string

class String is
  var data : access byte // i8*
  var length : i64

  this() is
    this.data := malloc(1ll)
    this.length := 0
  end

  this(literal : access byte) is
    this.data := literal
    this.length := strlen(literal)
    //this.length = strlen(literal)
  end

  method append(char : byte) is
    var len : i64 := this.length
    var newLen : i64 := len.Plus(1ll)

    var newData : access byte := malloc(newLen)
    strcpy(newData, this.data)

    var index : i64 := newLen.Minus(1ll)
    newData[index] := char

    strcpy(this.data, newData)

    this.length := newLen
  end

  method size() : i64 is
    return this.length
  end
end
```

Access types

Зачем? Методы.

```
1   %Car = type { %Vehicle, i32, i32 }
2
3   define void @Car_Create(ptr %thisCar) {
4   entry:
5     %thisCar1 = alloca ptr, align 8
6     store ptr %thisCar, ptr %thisCar1, align 8
7     %0 = load ptr, ptr %thisCar1, align 8
8     %Vehicle = getelementptr inbounds nuw %Car
9     %1 = alloca %Vehicle, align 8
10    call void @Vehicle_Create(ptr %1)
11    call void @llvm.memcpy.p0.p0.i64(ptr %Vehi
12    %2 = load ptr, ptr %thisCar1, align 8
13    %model = getelementptr inbounds nuw %Car,
14    store i32 2, ptr %model, align 4
15    ret void
16  }
```

Как?

```
82  // Pointer type
83  class TypeAccess : public Type {
84  public:
85    TypeAccess(const std::shared_ptr<Type> to)
86      : Type( kind: TYPE_ACCESS,  name: "access"), kind(ACC_GENERAL), to(to) {}
87
88    AccessKind kind;
89    std::shared_ptr<Type> to;
90
91    llvm::Type *toLLVMType(llvm::LLVMContext &lc) override {
92      auto toType = to->toLLVMType( [&] lc);
93      return llvm::PointerType::get(toType,  AddressSpace: 0);
94    }
95  };
```

If elif

Зачем? –

```
module ifelif

class Main is
  this() is
    var a : Integer := 2

    if a.Rem(3).Equal(0) then
      printf("mod 3\n")
    end
    else if a.Rem(2).Equal(0) then
      printf("mod 2\n")
    end
    else
      printf("no mod\n")
    end
  end
end
```

Как?

```
ifFalse
    -> ifSTMT
        ->ifFalse
            -> …
```

Modules

Зачем? STD

```
module string

class String is
  var data : access byte // i8*
  var length : i64

  this() is
    this.data := malloc(1ll)
    this.length := 0
  end

  this(literal : access byte) is
    this.data := literal
    this.length := strlen(literal)
    //this.length = strlen(literal)
  end
```

Как?

```
std::shared_ptr<ModuleDecl> Parser::parseProgram() {
  while (token->kind == TOKEN_MODULE_IMP) {
    sm.addIncludedModule(*buff, importedModuleName);

    globalSymbolTable->copySymbolFromModulesToCurrent(
        importedModuleName, // from
        moduleName          // to
    );

    globalTypeTable->importTypesFromModule(importedModuleName, to: moduleName);
```

```
    // LINK MODULES
    auto included :vector<FileData*> = sm.getIncluded(*buff);
    // sm.linkWithIncludedModules(*buff, );
    std::ranges::for_each(
      [>>] included,
      f: [&](auto &file) {
        if (llvm::Linker::linkModules( [&] *module, Src: std::move(file->module)))
          throw std::runtime_error("Failed to link modules");
        assert(module && "Module is null after linking!");
      }
    );
```

SizeOf

Зачем? –

```
12 module sizeof
11
10 class Main is
 9   this() is
 8
 7     var c : Real := 2.22
 6     var a : Integer := c as Integer
 5
 4     printf("a is %d with size %d\n", a, a.Size())
 3     printf("c is %f with size %d\n", c, c.Size())
 2
 1   end
13 end
```

```
[artjom@artjom cmake-build-release]$ ./sizeof.oout
a is 2 with size 4
c is 2.220000 with size 8
```

Как?

int size = (int)&((T*)0)[1]

```
%calltmp = call i32 (ptr, ...)
    @printf(ptr @0, i32 %a3,
    i64 ptrtoint (ptr getelementptr
        (i32, ptr null, i16 1) to i64))
```

```
llvm::Value* CodeGenVisitor::getSizeOfValue(llvm::Value* value) {
  llvm::Type* type = value->getType();
  llvm::Value* nullValue =
    llvm::ConstantPointerNull::get(
        T: llvm::PointerType::get(type,  AddressSpace: 0));
  llvm::Value* offset = builder->CreateGEP(
    type,
    nullValue,
    IdxList:  llvm::ConstantInt::get(
        [&] *context,
        V: llvm::APInt( numBits: 16,  val: 1)));
  return builder->CreatePtrToInt(offset, llvm::Type::getInt64Ty( [&] *context)
}
```

Main class

Зачем? –

```
12 module sizeof
11
10 class Main is
 9   this() is
 8
 7     var c : Real := 2.22
 6     var a : Integer := c as Integer
 5
 4     printf("a is %d with size %d\n", a, a.Size())
 3     printf("c is %f with size %d\n", c, c.Size())
 2
 1   end
13 end
```

```
[artjom@artjom cmake-build-release]$ ./sizeof.oout
a is 2 with size 4
c is 2.220000 with size 8
```

Как?

```
1  define i32 @main() {
2  entry:
3    %0 = alloca %Main, align 8
4    call void @Main_Create(ptr %0)
5    ret i32 0
6  }
```

Examples

```
module arrays

class Point is
  var x : Integer
  var y : Integer

  this(_x : Integer, _y : Integer) is
    this.x := _x
    this.y := _y
  end
end

class Main is
  this() is
    var arr : Array[Point, 5]

    printf("- Point array created: ")

    var i : Integer := 0
    for i, i.Less(5), i := i.Plus(1) is
      printf("%d %d | ", arr[i].x, arr[i].y)
    end

    // element manipulation
    arr[0] := Point(1, 2)
    var el : Point := arr[0]

    printf("\n new el: %d %d | arr[0] = %d %d\n", el.x, el.y, arr[0].x, arr[0].y)

    i := 0
    for i, i.Less(5), i := i.Plus(1) is
      printf("%d %d | ", arr[i].x, arr[i].y)
    end

    printf("\n\t WOW! \n")

  end
end
```

Examples

```
module inheritance

class Vehicle is
  var brand : Integer

  this() is
    this.brand := 1
  end

  method nuut() is
    printf("Nuut!\n")
  end

  method honk() is
    printf("Tuut, tuut!\n")
  end
end

class Car extends Vehicle is
  var model : Integer

  this() is
    this.Vehicle := Vehicle()
    this.model := 2
  end

  method honk() is
    printf("Overriden\n")
  end
end

class Main is
  this() is
    var v : Vehicle()
    var c : Car()

    v.honk()
    c.honk()
    c.nuut()

    printf("v brand = %d, c branhd = %d, c model = %d\n", v.brand, c.brand, c.model)
  end
end
```

Examples

```
module test

import string

class Main is
  this() is

    var str : String := String("Hello")

    printf("str = %s | length = %d\n", str.data, str.size())

    str.append('!')

    printf("str = %s | length = %d\n", str.data, str.size())

  end
end
```

```
module string

class String is
  var data : access byte // i8*
  var length : i64

  this() is
    this.data := malloc(1ll)
    this.length := 0
  end

  this(literal : access byte) is
    this.data := literal
    this.length := strlen(literal)
    //this.length = strlen(literal)
  end

  method append(char : byte) is
    var len : i64 := this.length
    var newLen : i64 := len.Plus(1ll)

    var newData : access byte := malloc(newLen)
    strcpy(newData, this.data)

    var index : i64 := newLen.Minus(1ll)
    newData[index] := char

    strcpy(this.data, newData)

    this.length := newLen
  end

  method size() : i64 is
    return this.length
  end
end
```

Examples

```
module ifelif

class Main is
  this() is
    var a : Integer := 2

    if a.Rem(3).Equal(0) then
      printf("mod 3\n")
    end
    else if a.Rem(2).Equal(0) then
      printf("mod 2\n")
    end
    else
      printf("no mod\n")
    end
  end
end
```

Исходный код

LLVMad/obewrong: Obewrong programming language

ИТМО

Спасибо за внимание!